Università
di Genova

Report High Performance Computing

Andrea Cattarinich 5137057

November 18, 2025

Contents

1 Architectures

For this project I have used two workstation, my personal one and the workstation provided by the University.

HP Envy 17-ae103nl (Laptop)

- **CPU**: Intel Core i7-6500U @ 2.50 GHz
- **Architecture**: x86_64
- **Core fisici**: 2
- **Thread logici**: 4 (Hyper-Threading)

University Workstation (Room 210)

- **CPU**: Intel Core i9-12900K (12th Gen)
- **Architecture**: x86_64
- **Core(s) per socket**: 16
- **CPU(s)**: 24
- **Thread(s) per core**: 2
- **Socket(s)**: 1
- **CPU max MHz**: 4.02 GHz
- **Profiling Tools**: Intel VTune Profiler, Intel Advisor

To use the Intel oneAPI tools, the environment was loaded with:

```
source /opt/intel/oneapi/setvars.sh
```

2 Amdahl's Law

This section discusses Amdahl's Law, a fundamental principle in parallel computing that helps predict the theoretical speedup in latency when using multiple processors.

2.1 Performance Metrics

Considering T_p the time to solve a problem using p processors, we define:

- **Speedup**: $S(p) = \frac{T_1}{T_p},\ 0 < S(p) \leq p$
- **Efficiency**: $E(p) = \frac{S(p)}{p},\ 0 < E(p) \leq 1$

2.2 The Law

Amdahl's Law can be expressed as a classical formula:

$$S(p) = \frac{1}{f_s + \frac{(1-f_s)}{p}}$$

where:

- $S(p)$: is the theoretical speedup
- f_s: is the proportion of sequential code (non-parallelizable)
- $1 - f_s$: is the proportion of parallelizable code
- p: is the number of processors

Ideal case $(f_s = 0)$

Implies that all code is parallelizable. The speedup becomes:

$$S(p) = \frac{1}{\frac{1}{p}} = p$$

Maximum speedup equals the number of processors. Each processor contributes fully to the speedup.

Theoretical limit case $(p \to \infty)$

By increasing the number of processors to infinity, the speedup reaches its theoretical limit:

$$\lim_{p \to \infty} S(p) = \frac{1}{f_s}$$

Even tough the number of processors increases indefinitely, the speedup is limited by the sequential portion of the code, encountering a bottleneck.

Real case $(f_s = \alpha p)$

Where α is a constant $\alpha <= 0.1$ representing the overhead per processor. The speedup becomes:

$$S(p) = \frac{1}{\alpha p + \frac{1-\alpha p}{p}} = \frac{p}{1 - \alpha p + \alpha p^2}$$

There is no free lunch: as we add more processors, the overhead increases linearly, impacting the overall speedup.

For α small: when α is small we do not have a significant overhead.

$$S(p) \approx p$$

For α large: when α is large, the overhead dominates. The more we add processors, the less benefit we get!

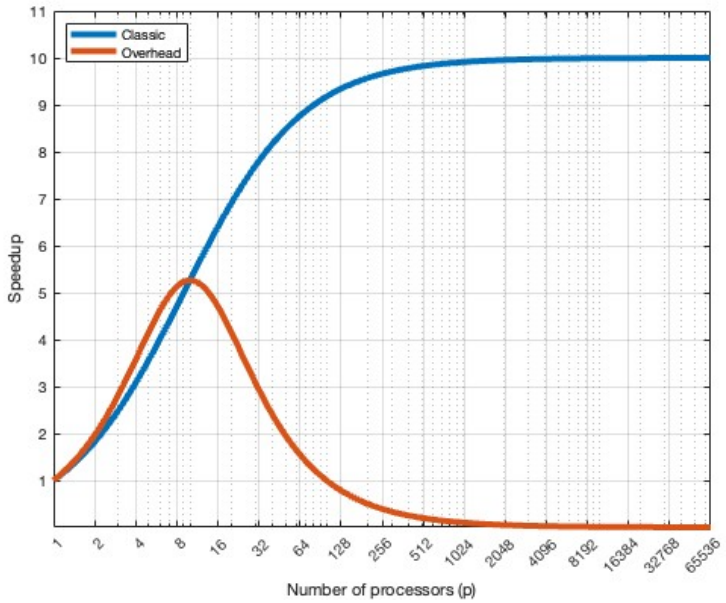
$$S(p) \approx \frac{p}{\alpha p^2} \approx \frac{1}{\alpha p}$$

2.3 Results

The following examples have been obtained using Matlab scripts are available in the repository (`classic_vs_overhead.m` and `parallel_comparison.m`).

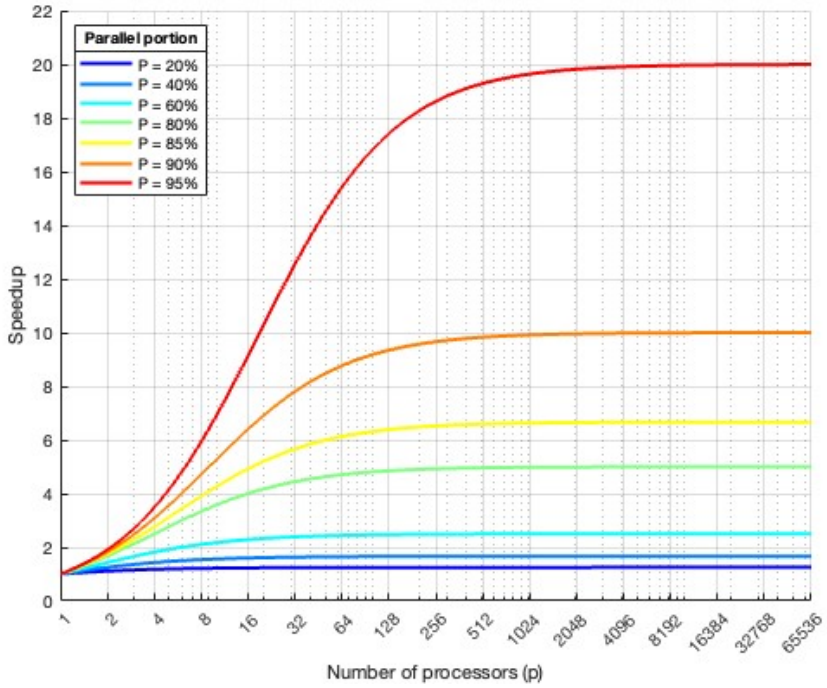
Classical vs Overhead

- The proportion of the parallelizable is 95% of code.
  $(f_s = 0.05,\ f_p = 1 - f_s = 0.95)$
- Overhead per processor
  $\alpha = 0.01$



Parallel speedup comparison

It is useful for understanding the limits of parallelization according to Amdahl's law: it highlights that simply increasing the number of processors is not enough if the sequential portion of the code is significant



3 Hotspot Analysis

The first step to improving the performance of a program is profiling. The goal of this analysis is to identify the parts of the program that consume the most execution time. To achieve this, I considered a data size resulting in a sequential execution time of ~ 30 seconds, by setting $N = 60'000$.

From this point onward, each executable is empirically measured over 10 runs to obtain reliable results.

3.1 Code description

```
int main() {
    int N = 100000;
    double *xr, *xi, *Xr, *Xi;

    fillInput(xr, xi, N);              // initialize input signal
    zero(Xr, Xi, N);                   // clear output buffers

    DFT(+1, xr, xi, Xr, Xi, N);        // compute DFT
    DFT(-1, Xr, Xi, xr, xi, N);        // compute IDFT

    checkResults(xr, xi, Xr, Xi, N); // verify correctness

    return 0;
}

void DFT(int idft, double* xr, ..., double* Xi, int N) {
    for (int k = 0; k < N; k++)
        for (int n = 0; n < N; n++) {
            double a = 2*M_PI*n*k/N;
            Xr[k] += xr[n]*cos(a) + idft*xi[n]*sin(a);
            Xi[k] += -idft*xr[n]*sin(a) + xi[n]*cos(a);
        }

    if (idft == -1)
        for (int n = 0; n < N; n++) {
            Xr[n] /= N;
            Xi[n] /= N;
        }
}
```

Listing 1: Pseudo code in C

The code implements the Discrete Fourier Transform (DFT) and its Inverse (IDFT) in C. It calculates the transform of a signal and verify the correctness through IDFT following the definition:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}, \quad k = 0, 1, \ldots, N-1$$

3.2 Sequential and Parallel Sections

In order to apply Amdahl's Law described in the previous section, we must identify the sequential and parallel parts of the code. I have empirically measured these sections by separating the serial and parallel portions. More details of how I obtained these results can be found in the code `sequential_parallel_code_identification.c`.

The results are the following:

Quantity	**Value**
Total execution time T_{tot}	32.48231 s
Time of the parallelizable section T_{par}	32.48021 s
Time of the serial section $T_{\text{ser}} = T_{\text{tot}} - T_{\text{par}}$	0.00209 s
Fraction of serial code $f_s = \frac{T_{\text{ser}}}{T_{\text{tot}}}$	0.006 %

These values will be used to estimate the theoretical speedup according to Amdahl's Law:

$$S(p) = \frac{1}{f_s + \frac{1-f_s}{p}}\Big|_{f_s=0.00006} = \frac{1}{0.00006 + \frac{1-0.00006}{p}} \approx p$$

Note: the smaller the serial fraction f_s, the higher the potential speedup achievable by parallelizing the main loop of the DFT.

Note: in the real case, the speedup is not linear. In particular, from the parallelization examples presented in Chapter 5, the speedup is approximately linear up to 8 threads; beyond this point, it deviates from linear behavior.

3.3 Compilation

Since the Intel(R) C++ Compiler Classic (ICC) is deprecated and was removed from product release in the second half of 2023. The Intel(R) oneAPI DPC++/C++ Compiler (ICX) was used to performe the analysis.

To identify the optimal sequential compilation strategy, I conduced testing across multiple Intel compiler variants (`ICC` and `ICX`), focusing on the vectorization report provided by the flag `-qopt-report`.
The program was compiled using the Intel compiler `ICX` with the following command (more details are provided in the `Makefile.seq`):

```
icx -O2 -vec -g \
    -fopenmp \
    -qopt-report=max \
    -qopt-report-file=reports/omp_homework.optrpt \
    -o build/omp_homework \
    src/omp_homework.c
```

The compilation flags have the following meaning:

- `-O2`: optimizations for better performance;

- `-fopenmp`: activates compilation for OpenMP directives;
- `-vec`: activates automatic vectorization of loops using SIMD instructions;
- `-g`: includes debugging information in the executable;
- `-qopt-report=max`: generates a detailed optimization report;
- `-qopt-report-file=...`: specifies the file where the optimization report is saved;
- `-o build/omp_homework`: sets the name and location of the compiled executable;
- `src/omp_homework.c`: specifies the source file to compile.

3.4 Hotspot Identification

The measurements were performed using two methods: **Manually** (using `omp_get_wtime()`) and **Profiler** (using `Intel VTune`)

Table 1: Manually

Function	Time (s)	Percentage (%)
fillInput	0.0020	0.0061
setOutputZero1	0.0000	0.0000
setOutputZero2	0.0000	0.0000
DFT	16.2665	50.0319
IDFT	16.2435	49.9613
Total	32.5122	100.0000

Table 2: Intel VTune Profiler

Function Stack	CPU Time: Total	Percentage (%)
Total	32.4300	100.0000
_start	32.4300	100.0000
__libc_start_main_impl	32.4300	100.0000
– main	32.4300	100.0000
— DFT	16.2420	50.0833
—— __svml_sincos2	9.3781	28.9180
—— __svml_sincos2_l9	2.7880	8.5969
— DFT	15.8800	48.9670
—— __svml_sincos2	9.7499	30.0645
—— __svml_sincos2_l9	2.4501	7.5549
— fillInput	0.3080	0.9497

Top Hotspots

This section lists the most active functions in your application. Optimizing these hotspot functions typically results in improving overall application performance.

Function	Module	CPU Time	% of CPU Time
__svml_cos2_l9	hotspots_identification	18.344s	56.6%
DFT	hotspots_identification	4.076s	12.6%
DFT	hotspots_identification	3.680s	11.3%
__svml_sin2_l9	hotspots_identification	3.386s	10.4%
__svml_sin2	hotspots_identification	1.852s	5.7%
[Others]	N/A*	1.092s	3.4%

*N/A is applied to non-summable metrics.

3.5 Conclusions

The main hotspot is the DFT() function and its inverse IDFT(), which together account for over 99,05% of the total execution time. All other functions have negligible impact. This indicates that **parallelization and vectorization efforts should focus on the double loop inside DFT()**.

The main hotspot is the loop contained in lines 72-80 (source: `src/omp_homework.c`). The dominant operations contributing to the algorithm's computational intensity are the `sin()` and `cos()` trigonometric functions, which account for the majority of the execution time, as shown in the following code.

```
// DFT/IDFT routine
// idft: 1 direct DFT, -1 inverse IDFT (Inverse DFT)
int DFT(int idft, double* xr, double* xi, double* Xr_o, double* Xi_o, int N){
  int k, n;
  for (k=0 ; k<N ; k++)
  {
    for (n=0 ; n<N ; n++)  {
      // Real part of X[k]
      Xr_o[k] += xr[n] * cos(n * k * PI2 / N) + idft*xi[n]*sin(n * k * PI2 / N);
      // Imaginary part of X[k]
      Xi_o[k] += -idft*xr[n] * sin(n * k * PI2 / N) + xi[n] * cos(n * k * PI2 / N);
    }
  }

  // normalize if you are doing IDFT
  if (idft==-1){
    for (n=0 ; n<N ; n++){
      Xr_o[n] /=N;
      Xi_o[n] /=N;
    }
  }

  return 1;
}
```

4 Vectorization

- TODO: Brief discussion about possible vectorization issues provided by the Intel compiler report.
- TODO: Aggiungere il flag `-open-simd` alla compilazione e vedere se cambia qualcosa
- TODO: altro???

5 Parallelization

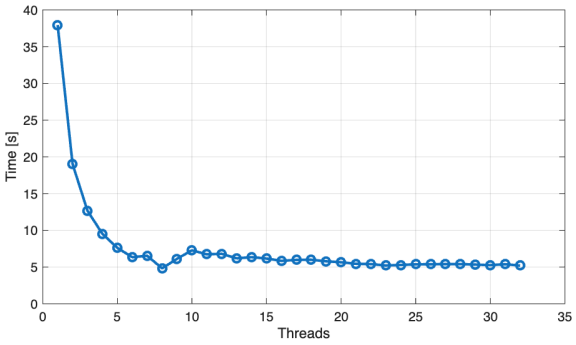
Ciclo `for` parallelizzato con OpenMP utilizzando `private(n)` per rendere ogni thread indipendente

```
#pragma omp parallel for private(n)
for (k=0 ; k<N ; k++)
{
    double Xr_temp = 0.0;
    double Xi_temp = 0.0;

    for (n=0 ; n<N ; n++)  {
        Xr_temp += ...
        Xi_temp += ...
    }

    Xr_o[k] = Xr_temp;
    Xi_o[k] = Xi_temp;
}
```

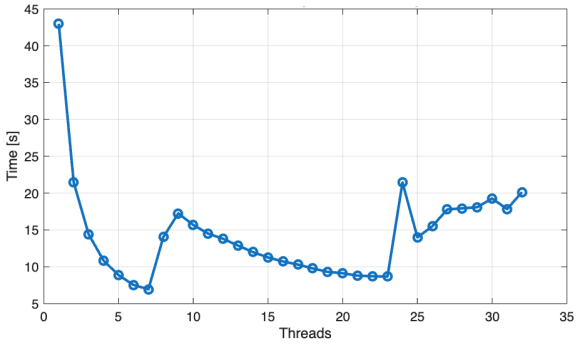
Ciclo `for` parallelizzato con OpenMP usando `reduction` per combinare i risultati dei thread.

```
int k, n;
for (k=0 ; k<N ; k++)
{
    double Xr_temp = 0.0;
    double Xi_temp = 0.0;

#pragma omp parallel for reduction(+:
    Xr_temp, Xi_temp)
    for (n=0 ; n<N ; n++)  {
        Xr_temp += ...
        Xi_temp += ...
    }

Xr_o[k] = Xr_temp;
Xi_o[k] = Xi_temp;
}
```

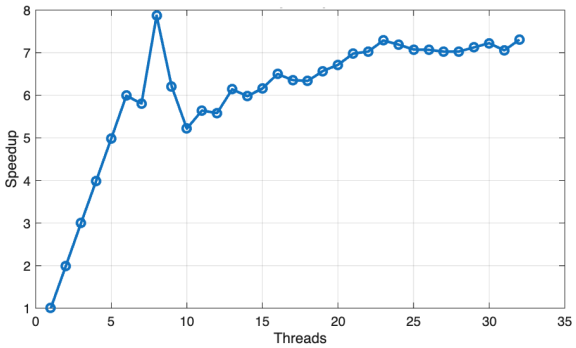
Execution time



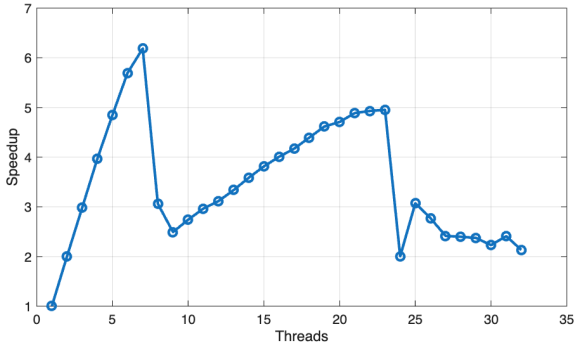
Execution time



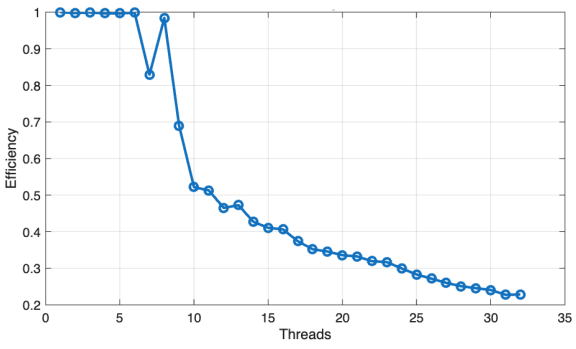
Speedup



Speedup



Efficiency



Efficiency